Case Study Guideline: Simultaneous Localisation and Mapping (SLAM) Algorithm for Autonomous Robots

September 17, 2026

1 Important

- This version of the guideline may be replaced, in whole or in part, by updates posted to Blackboard, either as new versions, as updates on the assignment page and/or as announcements posted to the Blackboard Announcements page. The version date appears at the title of case study guideline (this document) and on the Blackboard Assessment page.
- Please make sure you follow the latest guideline as it may affect what you are expected to turn in. No accommodation will be made for marks lost because you followed an outdated guideline.
- The written report will be checked through Turnitin. similarity with any existing materials (excluding cover page and references) should be less than **15%**, otherwise it will be treated as plagiarism.
- **Timeline**: the report is due at 23.59 pm, October 25th. The presentation will be held in the Week 14 tutorial or practical session. Each group will have a maximum of 30 minutes, comprising 15-25 minutes for the presentation and 5 minutes for a Q&A session.

2 Objective

In this case study, students will work in groups to select and study an algorithm for Simultaneous Localization and Mapping (SLAM) for an autonomous robot. This case study is designed to help students understand the theoretical principles behind SLAM algorithms and how they are implemented in practical applications.

The learning objectives are

- To gain a comprehensive understanding of a Simultaneous Localisation and Mapping (SLAM) algorithm.
- To learn how to implement a SLAM algorithm on an autonomous robot.
- To demonstrate the ability to connect theoretical concepts with practical implementations.
- To assemble essential software modules for autonomous systems using ROS2.
- To enhance teamwork and collaboration skills by working in groups.

3 Case Study Overview

3.1 Group Formation

Please discuss with classmates in your allocated practical or tutorial session, and form a group of 3-4 students. It is recommended the group should be balanced in terms of skills and expertise. Groups with fewer than 3 students will not be allowed, and groups with more than 4 students will require approval from the UC. Students who have not joined a group will be randomly allocated.

3.2 Algorithm Selection

There are many SLAM packages with source code are available. *OpenSLAM* provides a list of open source SLAM projects that you can choose from. This list is not exhaustive, and students are encouraged to explore and select alternative SLAM packages that may not be included. Nevertheless, it is the students' responsibility to ensure that the selected SLAM package is fully compatible with ROS or ROS 2. If ROS or ROS 2 is not used, a detailed description of the software framework, including the management of multiple threads/asks and the communication mechanisms among them, must be provided. Based on the content of this unit, I would like to recommend following SLAM packages for your reference, but again you can choose another SLAM package not listed here.

- *GMapping* - a highly efficient Rao-Blackwellized particle filer to learn grid maps from laser range data.
  - We have covered the theories in our lecture in details. If you need a safe case study, this is a good choice.
  - Reference papers:
    - `https://ieeexplore.ieee.org/document/4084563`
    - `https://ieeexplore.ieee.org/document/1570477`
  - Source code:
    - C++: `https://github.com/OpenSLAM-org/openslam_gmapping`
    - ROS2: `https://github.com/Project-MANAS/slam_gmapping`
    - ROS2: `https://github.com/siddarth09/ros2_gmapping`
- *Cartographer* - is an optimisation based SLAM algorithm developed by Google in 2016, and support was discontinued after 2019.
  - Reference paper or documents:
    - paper: `https://ieeexplore.ieee.org/document/7487258`.
    - documentation: `https://google-cartographer.readthedocs.io/en/latest/index.html`
    - A tutorial to run cartographer using ROS2 and Turtlebot3: `https://ros2-industrial-workshop.readthedocs.io/en/latest/_source/navigation/ROS2-Cartographer.html#`.
  - Source code:
    - C++: `https://github.com/cartographer-project/cartographer`
    - ROS: `https://github.com/cartographer-project/cartographer_ros`
- *SLAM-Toolbox*: The default SLAM solution for ROS 2.
  - This toolbox depends on many existing packages, such as *Open_karto*, *Ceres* optimiser, etc. It is a complicate toolbox and only minimal documents are provided, and if you decide to take this as your case study, a larger group is allowed.
  - Source code:
    - ROS2: `https://github.com/SteveMacenski/slam_toolbox`.
  - Documents
    - `https://joss.theoj.org/papers/10.21105/joss.02783`
    - `https://roscon.ros.org/2019/talks/roscon2019_slamtoolbox.pdf`

Each group should select a specific SLAM package for study, with justification by briefly explaining its key features, advantages and limitations, and suitable application scenarios. It is recommended that different groups choose different package to promote diversity and depth of exploration.

3.3 Research the Selected SLAM Algorithm

Find the origin of the selected SLAM algorithm, including the research papers that first introduced or later improved it. Begin by studying the original research paper to understand the core ideas and theoretical framework. Some research papers may omit implementation details, so it is essential to explore available implementations, particularly open-source projects, to gain a complete understanding of how the theoretical concepts are applied in practice. Students are required to

- Understand and present the underlying core theoretical principles that drive the algorithm, including (but not limited to) state estimation (sensor fusion) mathematical foundations (e.g., Bayesian filtering, graph optimisation), sensor models, motion prediction with/without odometry, scan matching methods, mapping and localisation processes, data association and loop closure mechanisms, etc.
- Present the theoretical workflow of the selected SLAM algorithm using visual representations (diagrams, flowcharts, or state-transition models), including (but not limited to)
  - Break down the algorithm into key stages, such as sensor data input, motion, prediction, data association or scan matching, and map update, loop closure detection and correction, optimisation (if applicable).
  - Identify data flow between stages.
  - Use clear visual elements, such as rectangles, arrows, diamonds, etc, and it should include the feedback loop.

  Please ensure that the workflow clearly outlines the theoretical details, including the sensor model and the specific types of data association or scan matching algorithms used. A generic SLAM workflow without these technical details will not be accepted.

3.4 Study and Analyse the SLAM Algorithm Implementation

- Create a clear architecture diagram of the SLAM implementation.
  - Students can use the theoretical workflow created above as the guideline to guide your code exploration.
  - Start from entry point, i.e., `main()` functions or equivalent entry scripts.
  - Identify which modules, classes, or libraries are initialised at the start.
  - Identify core modules that handle raw sensor data (LiDAR scans, odometry, IMU, camera images), pose graph composition, loop closure, and optimisation, and mapping managements.
  - For each modules identified above, describe the data flow and communication between modules (e.g., ROS nodes and topics, or function calls).
    - List the input and output signals of each module.
    - Use IDE features (e.g., "Go to Definition" in VS Code) to trace function calls. For examples, which the functions process sensor data, how motion prediction is conducted use odometry, how scan matching or feature extraction results are passed to mapping and localisation processes, etc.
    - ROS/ROS2 tools such as `rqt_graph`, etc can be used if a ROS/ROS2 implementation is available.
  - Generative AI tools are allowed here to help you to understand the code.
- Link code implementation with theories. For each core module listed in above architecture diagram
  - Analyse its practical implementation, and summarise it with pseudo codes. For example, the likelihood sensor module and its implementation provided in the practical 5 manual.
    - All variables should be defined and associate it with a physical meaning, like we did in the practical 5 manual.
    - Explanation of the pseudo code should be provided, diagram can be used if deemed necessary.
  - Provide code snippets, configuration files, and dependencies (if applicable) to illustrate the connection between algorithmic theory mentioned earlier and practical implementation.

Please note that while the theoretical overview of a SLAM algorithm and its implementation architecture are closely related, they serve different purposes. The theoretical overview focuses on what the algorithm does conceptually. It typically includes mathematical models, logical workflows, and algorithmic steps explained using formulas, pseudocode, or flowcharts. For example, an oversimplified theoretical overview could be:

> "The SLAM algorithm predicts the robot pose using odometry, corrects it using LiDAR scan matching, and updates the occupancy grid map while detecting loop closures via a graph-based optimisation."

In contrast, the implementation architecture describes how the algorithm is realized in code and executed in practice. This includes the structure of the code (modules, classes, packages, or nodes), the data flow between components, and the configuration of parameters. For example, an oversimplified description of an implementation architecture could be:

> "In the GMapping ROS package, the `slam_gmapping` node subscribes to `/scan` and `/odom`, calls the `ScanMatcher` module to align LiDAR data, updates the map in `GridSlamProcessor`, and publishes the resulting map on `/map`."

3.5 Application on an Autonomous Robot

Students are required to apply the SLAM algorithm to the simulated robots (ICTE4001Robot or virtual TurtleBot4) given in our tutorials, or the physical TurtleBot4 robot in the practical. **Otherwise, you will receive 0 mark for this part.**

- The SLAM algorithm should subscribe to the robot's right sensor topics, such as 2D scan, odometry, and any other relevant topics.
- Provide a brief evaluation of performance (map quality, localization accuracy, and observed limitations).
- Document instructions for reproduction, by providing
  - README file with environment setup and instructions.
  - ROS/ROS2 launch files or scripts for running the algorithm.
  - Config files with parameters.
  - Screenshots, maps, or video recordings of successful runs.
  - Notes on known limitations and troubleshooting.

3.6 Presentation [5 Marks]

Each group will prepare a comprehensive presentation on your selected SLAM algorithm and its implementation. The presentation will be held in the Week 13 tutorial or practical session. Each group will have a maximum of 30 minutes, comprising 15-25 minutes for the presentation and 5 minutes for a Q&A session. Groups should be prepared to answer questions from both peers and instructors.

The presentation should cover the following aspects:

- Introduction to SLAM.
  - Provide a brief overview of SLAM, its importance in autonomous robotics.
  - Describe the selected SLAM algorithm briefly, focusing on its applicability and use cases.
- Theoretical Overview and Foundations of the Selected SLAM Algorithm.
  - Present the overview of the algorithm using diagrams, flowcharts, or state-transition models to illustrate its data flow and main processes.
  - Explain the key components and their underlying theoretical principles of the chosen SLAM algorithm, for examples, state estimation, sensor models, data association, scan matching, mapping, and loop closure.
- Implementation Architecture Description.

  - Provide an overview of the algorithm's code or system architecture, highlighting major modules and their responsibilities, data flow between modules, and parameter configurations etc.
  - Match the theoretical workflow with the implementation architecture, explaining how each theoretical component is realised in code or system modules.

- Application Results and Performance Evaluation.
  - Demonstrate how you apply the selected SLAM algorithm to the simulated robot provided in the tutorials or the physical TurtleBot4 robot.
  - Discuss the performance in terms of map quality and localisation accuracy, or other related metrics.

3.7 Report [15 Marks]

Each group must submit a technical report that thoroughly documents the theoretical foundations, implementation analysis, and practical application of the selected SLAM algorithm, demonstrating a strong understanding of its algorithmic principles, implementation mechanisms, and integration into autonomous robot navigation.

The report should use following structure

- Title block.
  - Title of the case study (must include the name of the selected SLAM algorithm).
  - Group number and names/student IDs of all members should be included as well.
- Abstract (150 – 200 words)
  - A concise summary of the algorithm's purpose, theoretical foundations, implementation highlights, and main results.
- Introduction
  - Explain the importance of SLAM in autonomous robotics, highlighting its role in mapping and localisation.
  - Provide a concise technical overview of the selected SLAM algorithm, including key characteristics such as dimensionality (2D or 3D), map representation (feature-based or occupancy grid), core approach (filter-based or optimisation-based), and typical application scenarios.
- Research on the SLAM Algorithm.
  - Provide theoretical framework of the selected SLAM algorithm, such as, Kalman Filter, Particle Filter, Graph Optimisation, etc., including (but not limited to)
    * All core components of the SLAM framework, such as sensor model, motion model, map representation, data association, scan matching, probabilistic state estimation techniques (e.g., Bayesian filtering, particle filters, or graph optimisation).
    * Explain how to integrate all components together with the state estimation technique, and provide a theoretical workflow to visualise this process.
  - All variables in equations must be defined and explained.
  - Cite original and relevant research papers using IEEE style.
- Implementation Analysis
  - Describe the Software Architecture
    * Present and describe the required software architecture diagram that shows connected modules, and data flow between modules, and functions of each module.
  - Link the theories mentioned in "Research on SLAM Algorithm" to the Software Architecture, and list the pseudo code with detailed explanation.
  - List all dependencies, ROS packages, libraries, and optimisers used (e.g., Ceres, OpenKarto).
  - Provide details on important parameter settings (e.g., map resolution, sensor noise parameters, etc).
- Application Results and Performance Evaluation

- Describe the technical details of using Nav2 or other navigation package, and the selected SLAM package to achieve autonomous navigation of the robot in simulated environment provided in tutorials or physical TurtleBot4 robot.
- Discuss the of the algorithm's performance based on experiments on the simulated environments or other real environment with TurtleBot4.
- Insights gained from the project, including the practical implications of the algorithm in autonomous robotics.

Submit the report before 23.59 pm, October 25th. The report should be well-structured, properly referenced with IEEE style, and free from plagiarism.

4 Marking Rubrics

Table 1: Presentation Marking Rubric (Total: 5 Marks)

Criteria	Excellent (≥85%)	Good (70-85%)	Satisfactory (50-69%)	Unsatisfactory (<50%)
Introduction to SLAM (0.5 Mark)	Clear, concise overview of SLAM with strong explanation of relevance to autonomous robotics; the selected algorithm is introduced with context and relevance to use cases.	Adequate overview of SLAM and good introduction of the chosen algorithm, though minor details may be missing.	Basic explanation provided, but lacks depth or clarity in how SLAM or the algorithm applies to robotics.	Incomplete or unclear introduction, missing key concepts or relevance to autonomous robotics.
Theoretical Overview(1.5 Marks)	Presents a clear, well-structured theoretical explanation using diagrams/flowcharts; accurately identifies all key components (e.g., state estimation, mapping, loop closure); terminology is used correctly.	Good theoretical explanation with supporting visuals; most components are covered, with some minor gaps or inaccuracies.	Covers theoretical foundation but lacks clarity or depth; diagrams may be present but not well explained.	Poor or missing explanation; visual aids are unclear or absent; theoretical understanding is weak.
Implementation Architecture (1.5 Marks)	Detailed, coherent description of code/system architecture; clear mapping between theory and code; data flow, modules, and parameters are well explained.	Good description with some linkage to theoretical components; most modules and data flow explained with moderate clarity.	Partial or basic implementation explanation; limited connection to theory; architecture lacks clarity or depth.	Disorganised or unclear architecture explanation; poor linkage to theory; major components missing.
Application & Performance Evaluation (1 Mark)	Demonstrates successful implementation on the real TB4 or simulated TB4 or ICTE4001Robt in tutorial; integrates SLAM with Nav2 or other navigation system; evaluates map quality, localisation accuracy, and discusses limitations or insights gained.	Good demonstration with integration and performance discussion; some insights provided..	Basic demonstration shown; limited or vague discussion of integration or performance.	Application unclear or poorly explained; lacks evaluation and insight.
Delivery, Organisation, Q&A (0.5 Mark)	Presentation is clear, well-organised, and within time limit; speakers are confident and engaging; all group members contribute; able to answer questions accurately.	Good delivery and timing; minor issues with clarity or engagement; most members contribute; answers questions reasonably well.	Some issues with timing, clarity, or engagement; uneven participation; answers are vague or partially correct.	Poor presentation structure or delivery; limited engagement or preparation; cannot address questions.

Table 2: Report Marking Rubric (Total: 15 Marks)

Criteria	Excellent (≥85%)	Good (70-85%)	Satisfactory (50-69%)	Unsatisfactory (<50%)
Abstract (1 Mark)	Provides a concise and complete summary of the project: algorithm, theory, implementation, application, and results. Demonstrates clarity, precision, and insight in a short format.	Covers key areas but may lack completeness or precision. Clear but could be more concise or focused.	Communicates the general topic but lacks specificity or structure. May omit results or practical application.	Vague or incomplete; missing key outcomes or relevance.
Introduction (1 Mark)	Clearly defines the role of SLAM in autonomous robotics. Provides strong motivation for the selected algorithm with technical description (e.g., 2D/3D, feature/grid, optimisation/filter-based). .	Describes SLAM relevance and selected algorithm well, though may lack some technical precision or motivation.	Basic explanation provided; limited detail about algorithm characteristics or significance.	Introduction lacks clarity or technical focus; little or no context for algorithm selection.
Theoretical Framework (4 Marks)	Shows deep understanding of the SLAM algorithm's theoretical foundations. Includes accurate models, equations, and clearly defined variables. Concepts like filtering, sensor models, loop closure, etc., are well-integrated and connected. Visuals and citations used effectively.	Most theoretical components are explained clearly and correctly, with appropriate diagrams and citations. Minor conceptual or mathematical errors may exist.	General or fragmented explanation; some components are underdeveloped or oversimplified. Some errors or unclear logic.	Lacks theoretical understanding; poor explanations or missing components. Minimal or no use of diagrams and sources.
Software Architecture and Implementation (4 Marks)	Code structure is clearly and correctly interpreted. Theory is explicitly mapped to modules, classes, or ROS nodes. Pseudocode is well-written, correctly explained, and matches algorithmic purpose. Parameters and dependencies are well-documented and meaningfully discussed.	Architecture is mostly clear and reasonably linked to theory. Some parts may lack explanation or clarity. Pseudocode is provided but may not be consistently explained.	Partial or superficial code interpretation. Pseudocode is vague or mechanical. Poor linkage between theory and actual implementation.	Minimal explanation; structure and pseudocode unclear or incorrect; no effort to explain parameter settings or practical code logic.
Application and Performance Evaluation (3 Marks)	Strong demonstration of working SLAM+Nav2 integration in simulation or on TurtleBot4. Results (maps, screenshots, video) are clearly presented. Performance evaluation is insightful and critically reflective, discussing strengths, weaknesses, and practical implications.	Demonstrates good application with some documentation and analysis. Integration and evaluation are present but lack critical depth.	Basic demonstration without clear integration with Nav2 or navigation framework. Performance discussion is vague or generic.	No clear implementation; minimal or no evaluation; unclear how the SLAM algorithm was used or tested.
Report Structure, Writing, and Referencing (2 Marks)	Well-organised, technically written, and easy to follow; correct IEEE-style referencing; includes labelled figures and tables.	Good structure and mostly correct citations; minor issues in writing or formatting.	Adequate structure; referencing style inconsistent; some language or clarity issues.	Poor writing, structure, or referencing; difficult to follow.